UNIVERSIDADE FEDERAL DE UBERLÂNDIA - UFU

FACULDADE DE COMPUTAÇÃO - FACOM

Disciplina: Estruturas de Dados | Professor(a): ED 2026

DOCUMENTAÇÃO TÉCNICA E CONCEITUAL

Sistema Clínico VidaSaudável: Integração das Quatro Estruturas Lineares Clássicas em C

Resumo

Este documento apresenta uma análise conceitual, arquitetural e prática da implementação do sistema de atendimento hospitalar **Clínica VidaSaudável**, desenvolvido em linguagem C. O projeto atende integralmente aos requisitos curriculares de domínio das quatro estruturas de dados lineares clássicas: **Lista Simplesmente Encadeada com Cabeça** (cadastro de pacientes), **Lista Circular** (escala rotativa dos médicos), **Fila FIFO** (sala de espera) e **Pilha LIFO** (histórico de atendimentos reversível). São detalhados os conceitos de ponteiros simples e duplos, alocação dinâmica com gerenciamento rigoroso de memória, encapsulamento de Tipos Abstratos de Dados (TAD) e a mecânica interna de cada função implementada.

Sumário

1 Objetivo Pedagógico e Competências Demonstradas **3**

2 Contexto e Arquitetura da Clínica VidaSaudável **3**

2.1 As Quatro Coleções Independentes . . . 3
2.2 Fluxo Típico de Atendimento . . . 4
2.3 Resolução de Conflitos de Nomes e Engenharia dos Cabeçalhos (`.h`) . . . 5
2.4 Lista Encadeada Simples com Cabeça (`lista_com_cabeca.c/.h`) . . . 6
2.4.1 Conceito do Nó Cabeça (Sentinela) . . . 6
2.4.2 Implementação das Funções Chave . . . 6
2.5 Lista Circular Simplesmente Encadeada (`lista_circular.c/.h`) . . . 8
2.5.1 Conceito do Ponteiro para a Cauda (`*p`) . . . 8
2.5.2 Implementação das Funções da Lista Circular . . . 8
2.6 TAD Fila — FIFO (`fila.c/.h`) . . . 10
2.6.1 Encapsulamento e Composição . . . 10
2.7 TAD Pilha — LIFO (`pilha.c/.h`) . . . 10
2.7.1 Encapsulamento e Histórico Reversível . . . 10

3 O Sistema Integrador (`clinica.c`) 12
3.1 Análise das Operações de Integração 12
3.1.1 Atendimento de Paciente (`Opção 10`) 12
3.1.2 Desfazer Último Atendimento (`Opção 11`) 12
3.1.3 Encerramento Seguro e Liberação de Memória (`Opção 0`) 13

4 Rastreamento Passo a Passo de um Cenário Real 14

5 Conclusão 14

1 Objetivo Pedagógico e Competências Demonstradas

O objetivo central deste trabalho prático é demonstrar a capacidade de projetar, modularizar e integrar em um sistema interativo coeso as quatro estruturas de dados lineares clássicas em linguagem C.

Mapeamento dos Requisitos da Atividade

1. **Domínio de Ponteiros e Memória Dinâmica**:
   - Uso de ponteiros simples (`T*`) para travessia, busca e leitura de estruturas.
   - Uso de ponteiros duplos (`T**`) para funções modificadoras onde o nó inicial ou a cabeça da coleção pode mudar na função chamadora.
   - Alocação explícita na Heap via `malloc(sizeof(...))` e liberação completa com `free()`, evitando vazamentos de memória (*memory leaks*).
2. **Encapsulamento e Tipos Abstratos de Dados (TAD)**:
   - Separação estrita entre interface (`.h`) e implementação (`.c`).
   - Composição de estruturas de alto nível (`Fila` e `Pilha`) aproveitando internamente as listas encadeadas sem expor seus nós para o código cliente.
3. **Comportamentos Lineares Distintos**:
   - **FIFO (First-In, First-Out)**: Fila de espera na recepção médica.
   - **LIFO (Last-In, First-Out)**: Pilha para permitir a operação de "Desfazer Atendimento" (*Undo*).
   - **Circular**: Escala contínua e cíclica de plantonistas.
   - **Linear com Sentinela/Cabeça**: Cadastro permanente de pacientes com busca e remoção arbitrária por ID.

2 Contexto e Arquitetura da Clínica VidaSaudável

O sistema hospitalar **Clínica VidaSaudável** gerencia o fluxo de pacientes desde o acolhimento até a consulta com o médico plantonista, registrando auditoria reversível de eventos.

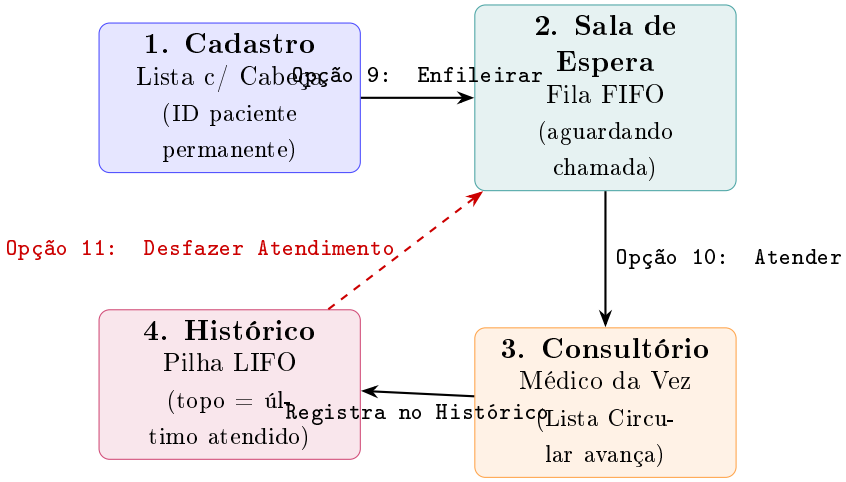
2.1 As Quatro Coleções Independentes

Coleção / Módulo	Estrutura Subjacente	Disciplina	Papel no Sistema	Complexidade
Pacientes Cadastrados	Lista Encadeada c/ Cabeça	Linear com Sentinela	Cadastro permanente com ID único	Busca: $O(n)$, Rem: $O(n)$
Escala de Médicos	Lista Circular Simples	Anel Cíclico	Plantonistas rotativos	Inserir/Avançar: $O(1)$
Sala de Espera	Fila (TAD Fila)	FIFO (Primeiro a Chegar)	Ordem de atendimento médico	Push/Pop: $O(1)$
Histórico Atendimento	Pilha (TAD Pilha)	LIFO (Último Atendido)	Desfazer o último atendimento	Push/Pop: $O(1)$

Tabela 1: Quadro Comparativo das Coleções e Estruturas de Dados

2.2 Fluxo Típico de Atendimento

O diagrama abaixo ilustra o ciclo operacional completo entre as quatro estruturas:



1. **Cadastro**: Paciente é inserido no cadastro permanente (`Opção 1`).
2. **Enfileiramento**: O paciente cadastrado dá entrada na sala de espera (`Opção 9`).
3. **Atendimento**: O próximo da fila é chamado pelo médico plantonista da vez (`Opção 10`). O paciente é desenfileirado e empilhado no histórico; o médico plantonista da escala circular avança para o próximo da lista.
4. **Desfazer Atendimento**: Se o médico precisa cancelar ou corrigir o atendimento (`Opção 11`), o paciente no topo da pilha é desempilhado e devolvido à fila de espera.

2.3 Resolução de Conflitos de Nomes e Engenharia dos Cabeçalhos (`.h`)

Em linguagem C pura, não existem *namespaces* (como em C++ ou C#). Todas as funções globais e definições de `structs`/ `typedefs` compartilham um único espaço global de nomes. Quando múltiplos arquivos de cabeçalho definem tipos genéricos iguais (ex: `No`) ou funções com nomes idênticos (ex: `criar_lista()`), a inclusão simultânea desses arquivos em um único programa integrador como `clinica.c` dispara erros fatais de redefinição no compilador GCC e colisão de símbolos no *linker*.

Para resolver esse problema e permitir que as quatro estruturas convivam harmonicamente no mesmo programa, realizamos uma refatoração arquitetural nos quatro arquivos de cabeçalho:

1. `lista_com_cabeca.h` **(Lista para Pacientes e Pilha)**:
   - O tipo da estrutura foi renomeado de `No` para `ListaNo`:

```c
typedef struct no_lista {
  int dado;
  struct no_lista *prox;
} ListaNo;
```

   - As funções ganharam o prefixo exclusivo `lista_`: `lista_criar_lista()`, `lista_adicionar_inicio()`, `lista_buscar_valor()`, `lista_remover_valor()`, `lista_tamanho_lista()`.
2. `lista_circular.h` **(Lista para Médicos e Fila)**:
   - O tipo da estrutura foi renomeado para `CircularNo`:

```c
typedef struct no_circular {
  int dado;
  struct no_circular *prox;
} CircularNo;
```

   - Adicionou-se a função `avancar_lista(CircularNo **L)`, fundamental para rotacionar a escala dos médicos plantonistas sem destruir nós.
3. `fila.h` **(Sala de Espera - FIFO)**:
   - Declara explicitamente a composição interna usando a lista circular:

```c
typedef struct {
  int tam;
  CircularNo *L; // Composicao interna opaca
} Fila;
```

   - Expõe apenas a interface limpa do TAD: `fila_adicionar()`, `fila_topo()`, `fila_remover()`, `fila_tamanho()`.
4. `pilha.h` **(Histórico Reversível - LIFO)**:
   - Declara explicitamente a composição interna usando a lista com cabeça:

```c
typedef struct {
  ListaNo *L; // Composicao interna opaca
} Pilha;
```

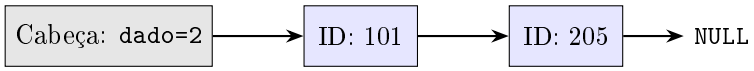
   - Expõe apenas a interface limpa do TAD: `pilha_adicionar()`, `pilha_topo()`, `pilha_remover()`, `pilha_tamanho()`.

2.4 Lista Encadeada Simples com Cabeça (`lista_com_cabeca.c/.h`)

2.4.1 Conceito do Nó Cabeça (Sentinela)

A lista utiliza um **nó cabeça** (ou nó sentinela) alocado na inicialização. Esse nó nunca contém dados de paciente; em vez disso:

- `head->dado` armazena a **quantidade total de elementos** da lista ($O(1)$ para tamanho).
- `head->prox` aponta para o primeiro elemento real, ou `NULL` se a lista estiver vazia.
- **Vantagem técnica**: Elimina o caso especial de inserir ou remover no início. O nó anterior ao primeiro sempre existe: é o próprio nó cabeça!



2.4.2 Implementação das Funções Chave

```c
ListaNo* lista_criar_lista() {
  ListaNo *q = (ListaNo*) malloc(sizeof(ListaNo));
  q->dado = 0;     // Contador de tamanho
  q->prox = NULL; // Nenhum paciente ainda
  return q;
}

void lista_adicionar_final(ListaNo *p, int x) {
  ListaNo *q, *aux;
  q = (ListaNo*) malloc(sizeof(ListaNo));
  q->dado = x;
  q->prox = NULL;

  aux = p; // Comeca a partir do no cabeca
  while (aux->prox != NULL)
    aux = aux->prox; // Caminha ate o ultimo no existente

  aux->prox = q; // Encadeia o novo no no fim
  p->dado++;      // Incrementa o contador na cabeca
}
```

Listing 1: Criação e Inserção no Fim com Nó Cabeça

Por que passar `ListaNo *p` em vez de `ListaNo **p`?

Como o nó cabeça foi alocado na criação da lista e seu endereço na memória Heap **nunca se altera** durante a vida útil do cadastro, a função recebe apenas `ListaNo *p`. O ponteiro da cabeça permanece fixo; apenas o encadeamento dos campos internos (`aux->prox`) e o valor do contador (`p->dado`) são alterados.

```c
void lista_remover_valor(ListaNo *p, int v) {
  ListaNo* q = p; // q e o predecessor do no a ser verificado
  while (q->prox != NULL){
    if(q->prox->dado == v) break;
    q = q->prox;
  }
  if(q->prox == NULL) return; // Paciente nao encontrado

  ListaNo* tmp = q->prox;      // Guarda ponteiro do no a desalocar
  q->prox = tmp->prox;         // 'Pula' o no a ser removido
  free(tmp);                   // Libera a memoria da Heap
```

```
12    p->dado--;                    // Atualiza o contador na cabeca
13 }
```

Listing 2: Remoção Segura por Valor

2.5 Lista Circular Simplesmente Encadeada (`lista_circular.c/.h`)

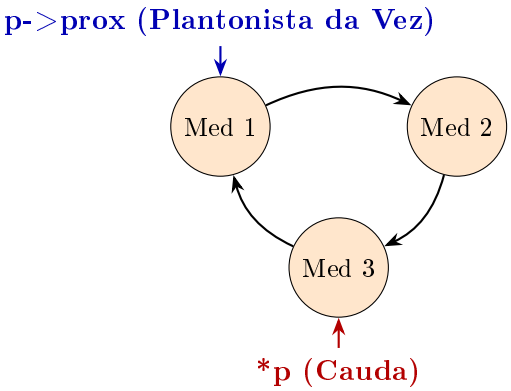
2.5.1 Conceito do Ponteiro para a Cauda (`*p`)

Diferente de uma lista linear onde o último nó aponta para `NULL`, na lista circular o último nó aponta de volta para o primeiro:

$$\text{último} \rightarrow \text{prox} = \text{primeiro} \tag{1}$$

A implementação adota a técnica mais eficiente da ciência da computação para listas circulares: **manter um único ponteiro para o último nó** (cauda `p`):

- O último elemento é acessado em $O(1)$: `p`.
- O primeiro elemento (o plantonista da vez) é acessado em $O(1)$: `p->prox`.
- A rotação de plantão (`avancar_lista`) é uma simples atribuição em tempo constante $O(1)$: `*p = (*p)->prox`!



2.5.2 Implementação das Funções da Lista Circular

```c
// Avanca o plantao da escala medica em tempo O(1)
void avancar_lista(CircularNo **p) {
  if (*p != NULL)
    *p = (*p)->prox; // A cauda vira o proximo no; o inicio vira o seguinte!
}

// Retorna o ID do medico plantonista da vez em O(1)
int acessar_primeiro(CircularNo *p){
  if(p == NULL) return -1; // Lista vazia
  CircularNo *q = p->prox; // p->prox e sempre o primeiro no!
  return q->dado;
}

// Insere novo medico no fim da escala
void adicionar_final(CircularNo **p, int x) {
  CircularNo *q = (CircularNo*) malloc(sizeof(CircularNo));
  q->dado = x;
  if(*p == NULL) { // Lista vazia: no aponta para si proprio
    *p = q;
    q->prox = q;
  } else {
    q->prox = (*p)->prox; // O novo no aponta para o primeiro no
    (*p)->prox = q;       // A antiga cauda aponta para o novo no
    *p = q;               // O ponteiro da cauda passa a ser o novo no!
  }
}
```

Listing 3: Avanço de Plantão e Inserção no Fim em $O(1)$

Por que `CircularNo **p` (Ponteiro Duplo)?

Quando a lista circular está vazia, o ponteiro na função `main` vale `NULL`. Para que a função `adicionar_final` altere o valor do ponteiro da `main`, ela precisa receber o **endereço daquele ponteiro** (`&medicos`). O desreferenciamento `*p = q` atualiza a variável original na memória da `main`.

2.6 TAD Fila — FIFO (`fila.c/.h`)

2.6.1 Encapsulamento e Composição

O TAD `Fila` encapsula a complexidade dos nós encadeados. Seu tipo de dado exposto na interface é:

```
typedef struct {
  int tam;          // Quantidade de pacientes na sala de espera
  CircularNo *L;    // Lista circular interna usada como buffer FIFO
} Fila;
```

Vantagem Arquitetural da Fila sobre Lista Circular

Ao utilizar a lista circular internamente:

- **Enfileirar** (`fila_adicionar`): chama `adicionar_final(&L, x)` em $O(1)$.
- **Espiar Primeiro** (`fila_topo`): chama `acessar_primeiro(L)` em $O(1)$.
- **Desenfileirar** (`fila_remover`): chama `remover_inicio(&L)` em $O(1)$.

Garante-se o comportamento estrito **FIFO** (Primeiro a Entrar, Primeiro a Sair) com todas as operações essenciais executadas em tempo constante $O(1)$ sem desperdício de memória!

```
Fila* fila_criar() {
  Fila *p = (Fila*) malloc(sizeof(Fila));
  p->tam = 0;
  p->L = criar_lista(); // Inicializa lista circular interna
  return p;
}

void fila_adicionar(Fila **p, int x) {
  adicionar_final(&((*p)->L), x); // Entra no fim da fila
  (*p)->tam++;
}

void fila_remover(Fila **p) {
  if ((*p)->tam == 0) return;
  remover_inicio(&((*p)->L));       // Sai do inicio da fila
  (*p)->tam--;
}

int fila_topo(Fila *p) {
  return acessar_primeiro(p->L);    // Proximo da fila a ser atendido
}
```

Listing 4: Operações da Fila FIFO

2.7 TAD Pilha — LIFO (`pilha.c/.h`)

2.7.1 Encapsulamento e Histórico Reversível

O TAD `Pilha` armazena o histórico dos atendimentos médicos realizados:

```
typedef struct {
  ListaNo *L; // Lista encadeada com cabeca usada como buffer LIFO
} Pilha;
```

O topo da pilha coincide com a primeira posição logo após o nó cabeça da lista encadeada (`L->prox`):

- **Empilhar** (`pilha_adicionar`): insere no início da lista via `lista_adicionar_inicio(p->L, x)` em $O(1)$.
- **Topo** (`pilha_topo`): consulta o primeiro nó via `lista_acessar_primeiro(p->L)` em $O(1)$.
- **Desempilhar** (`pilha_remover`): remove o primeiro nó via `lista_remover_inicio(p->L)` em $O(1)$.

O elemento inserido mais recentemente fica no topo. Quando o usuário executa a opção de "Desfazer Atendimento", a operação de desempilhar (*pop*) devolve exatamente o último paciente consultado!

3 O Sistema Integrador (`clinica.c`)

O arquivo `clinica.c` atua como o controlador da aplicação. Na função `main`, as quatro coleções são instanciadas independentemente:

```c
int main(void) {
  ListaNo *pacientes   = lista_criar_lista(); // 1. Cadastro permanente
  CircularNo *medicos  = criar_lista();       // 2. Escala de plantao
  Fila *espera         = fila_criar();        // 3. Sala de espera (FIFO)
  Pilha *historico     = pilha_criar();       // 4. Historico (LIFO)
  int opcao;
  // ... loop interativo ...
```

Listing 5: Instanciação das Quatro Estruturas no main

3.1 Análise das Operações de Integração

3.1.1 Atendimento de Paciente (`Opção 10`)

Esta é a operação mais rica do sistema, pois coordena simultaneamente três coleções distintas:

```c
case 10:
  if (fila_tamanho(espera) == 0)
    printf("Sala de espera vazia.\n");
  else if (tamanho_lista(medicos) == 0)
    printf("Nao ha medico de plantao.\n");
  else {
    int paciente = fila_topo(espera);          // 1. Identifica o proximo da fila
    int medico   = acessar_primeiro(medicos);  // 2. Identifica o plantonista da vez

    fila_remover(&espera);                     // 3. Paciente SAI da sala de espera (FIFO)
    pilha_adicionar(&historico, paciente);     // 4. Paciente ENTRA no historico (LIFO)

    printf("Paciente %d atendido pelo medico %d.\n", paciente, medico);
    avancar_lista(&medicos);                   // 5. Escala circular avanca para o prox medico!
  }
  break;
```

Listing 6: Coordenação entre Fila, Pilha e Lista Circular

3.1.2 Desfazer Último Atendimento (`Opção 11`)

Demonstra a utilidade prática da estrutura de Pilha (LIFO) para histórico reversível:

```c
case 11:
  if (pilha_tamanho(historico) == 0)
    printf("Nao ha atendimento para desfazer.\n");
  else {
    int paciente = pilha_topo(historico);      // 1. Consulta o ultimo atendido
    pilha_remover(&historico);                 // 2. Desempilha do historico
    fila_adicionar(&espera, paciente);         // 3. Devolve paciente para a fila de espera!
    printf("Atendimento do paciente %d desfeito; voltou para a fila.\n",
    paciente);
  }
  break;
```

Listing 7: Reversão de Atendimento via Desempilhamento

3.1.3 Encerramento Seguro e Liberação de Memória (`Opção 0`)

Ao finalizar a aplicação, todas as quatro coleções são desalocadas recursiva ou iterativamente, garantindo vazamento zero de memória (*Zero Memory Leaks*):

```
lista_destruir_lista(&pacientes);
destruir_lista(&medicos);
fila_destruir(&espera);
pilha_destruir(&historico);
return 0;
```

Listing 8: Desalocação Completa na Saída

4 Rastreamento Passo a Passo de um Cenário Real

A tabela a seguir demonstra a evolução das quatro estruturas de dados durante uma sequência de ações típica no terminal:

Passo	Ação do Usuário	Cadastro (`ListaNo`)	Médicos (`CircularNo`)	Fila (`Fila`)	Pilha (`Pilha`)
1	Cadastrar pac. 101 e 102	$[101, 102]$	$\emptyset$	$\emptyset$	$\emptyset$
2	Cadastrar méd. 1 e 2	$[101, 102]$	$[1 \circlearrowleft 2]$	$\emptyset$	$\emptyset$
3	Colocar 101 na fila	$[101, 102]$	$[1 \circlearrowleft 2]$	$[101]$	$\emptyset$
4	Colocar 102 na fila	$[101, 102]$	$[1 \circlearrowleft 2]$	$[101, 102]$	$\emptyset$
5	**Atender paciente**	$[101, 102]$	$[2 \circlearrowleft 1]$ (avançou!)	$[102]$	$[\mathbf{101}]$ (topo)
6	**Atender paciente**	$[101, 102]$	$[1 \circlearrowleft 2]$ (avançou!)	$\emptyset$	$[\mathbf{102}, 101]$
7	**Desfazer atendimento**	$[101, 102]$	$[1 \circlearrowleft 2]$	$[102]$	$[\mathbf{101}]$

Tabela 2: Trace de Estados da Memória durante o Fluxo Clínico

Observações do Rastreamento

- **No passo 5**: O paciente 101 é atendido pelo médico 1 (primeiro da escala). A fila passa a ter apenas o 102 (disciplina FIFO). A escala médica avança e o médico 2 assume o plantão da vez.
- **No passo 6**: O paciente 102 é atendido pelo médico 2. A pilha recebe 102 no topo (ficando $[102, 101]$). A escala circular completa a volta e o médico 1 torna a ser o da vez.
- **No passo 7**: Ao desfazer, o paciente 102 (que estava no topo da pilha por ter sido o último atendido) é retirado da pilha e recolocado na fila de espera. O histórico volta a conter apenas o paciente 101.

5 Conclusão

O projeto **Clínica VidaSaudável** cumpre integralmente os requisitos fundamentais da disciplina de Estruturas de Dados:

1. **Domínio Algorítmico**: Aplicação das complexidades $O(1)$ nas operações de inserção/remoção em filas, pilhas e rotação de plantão circular.
2. **Engenharia de Software em C**: Alta modularidade, isolamento de TADs via arquivos de cabeçalho e ocultação da representação interna.
3. **Robustez Operacional**: Tratamento correto de entradas do usuário, prevenção de operações em coleções vazias e destruição completa dos nós dinâmicos da Heap.